Traffic Navigation System using AI

Project Title: Traffic Navigation System using AI

Course: Build Your Own Project (BYOP)

Student Name: Faleha Karim

Registration Number: 25BCE11264

Institution: VIT University

Submission Date: 31 March 2026

Problem Statement

Traffic congestion has become a significant issue in modern urban environments due to rapid population growth and increased vehicle usage. This leads to longer travel times, increased fuel consumption, and environmental pollution. Existing navigation systems often rely on static or semi-dynamic data and may not efficiently adapt to real-time obstacles such as roadblocks, accidents, or heavy traffic conditions.
This project aims to design and implement an AI-based Traffic Navigation System that intelligently determines the most efficient path between a source and destination. The system utilizes classical AI search algorithms to simulate decision-making in a grid-based environment. It considers obstacles, explores alternative paths, and ensures optimal route selection based on the chosen algorithm.

Functional Requirements

The system provides several functionalities to ensure effective pathfinding and user interaction:

●User Capabilities:

Ability to select starting and destination nodes within the grid

Option to add or remove obstacles, simulating blocked roads

●Selection of different algorithms such as:

Breadth-First Search (BFS)

Depth-First Search (DFS)

A* Search Algorithm

●System Capabilities:

Automatically generate a grid-based environment

Perform path computation using the selected algorithm

Display step-by-step traversal (optional visualization)

Provide final output via Command Line Interface (CLI)

Handle edge cases such as unreachable destinations

Non-Functional Requirements

These requirements ensure the system performs efficiently and remains user-friendly:

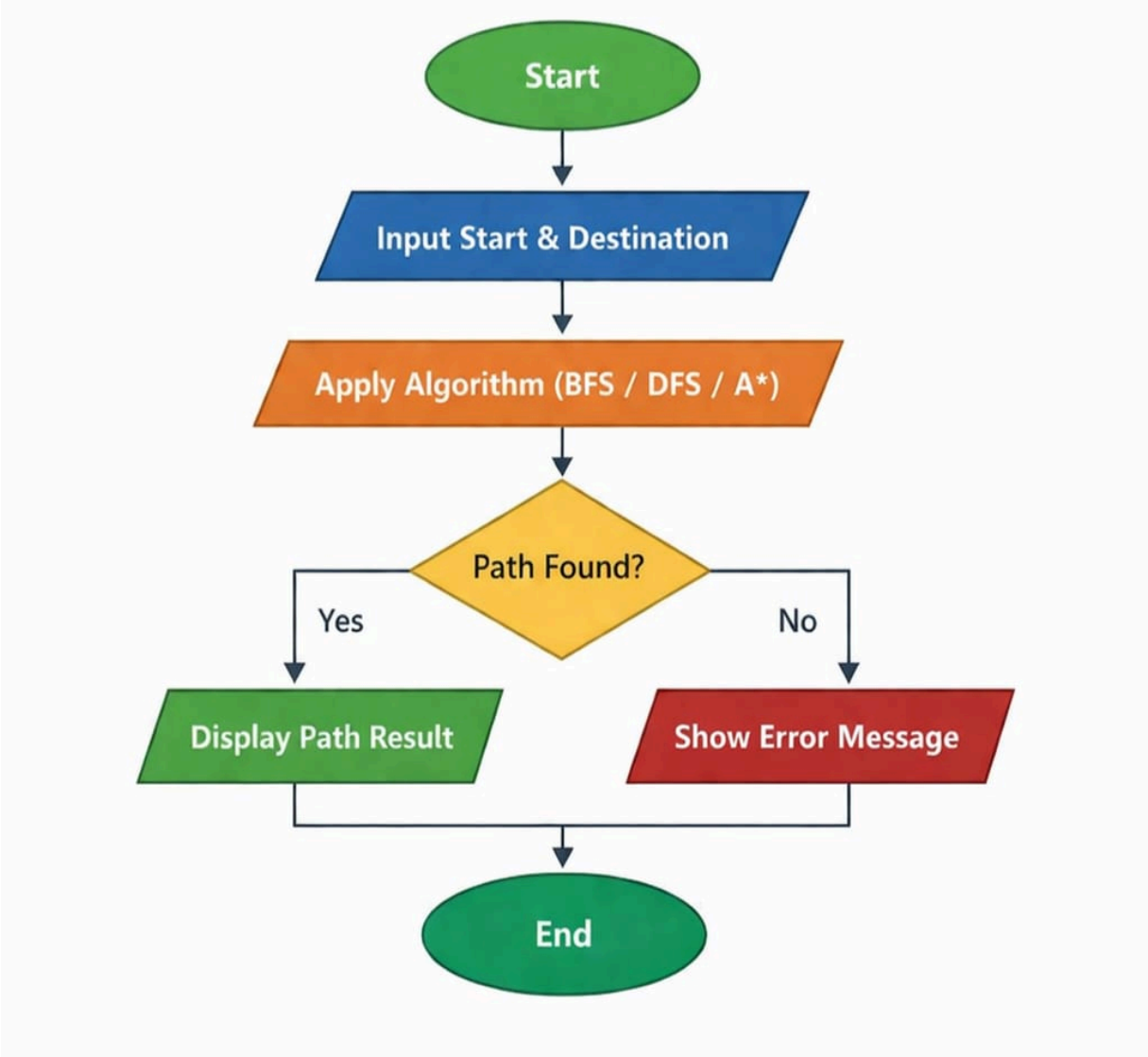
- Performance: The system should compute paths quickly even for larger grids
- Usability: Interface should be simple, requiring minimal user input
- Reliability: Algorithms must consistently produce correct results
- Scalability: System should handle increased grid sizes without major performance degradation
- Maintainability: Code should be modular, readable, and easy to update

System Architecture

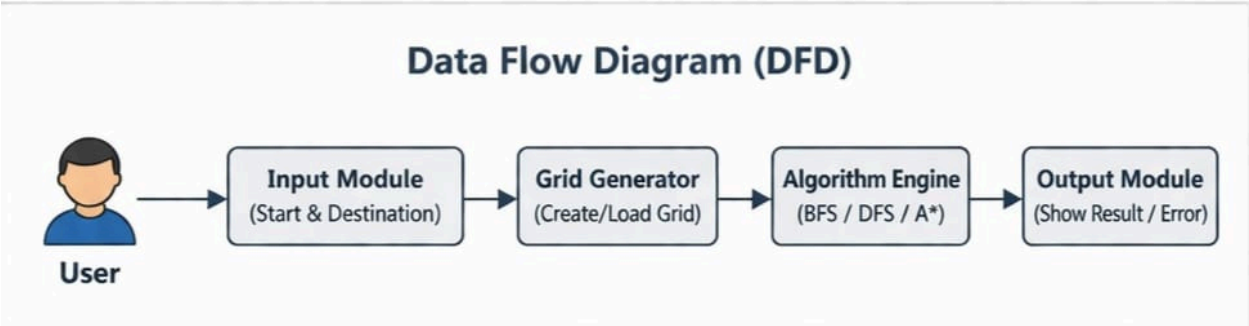
●Components:
Input Module – Accepts user inputs such as start/end nodes and obstacles
●Grid Generator – Creates a structured grid representing the map
●Pathfinding Engine – Executes selected algorithms (BFS, DFS, A*)
●Output Module – Displays the resulting path and status
●Workflow:
User provides input
Grid is generated with obstacles
Selected algorithm processes the grid
Shortest/valid path is computed
Result is displayed to the user
● Dual Interface Support: The system supports both GUI-based visualization and CLI-based execution for flexibility and evaluation compatibility.

Design Diagrams

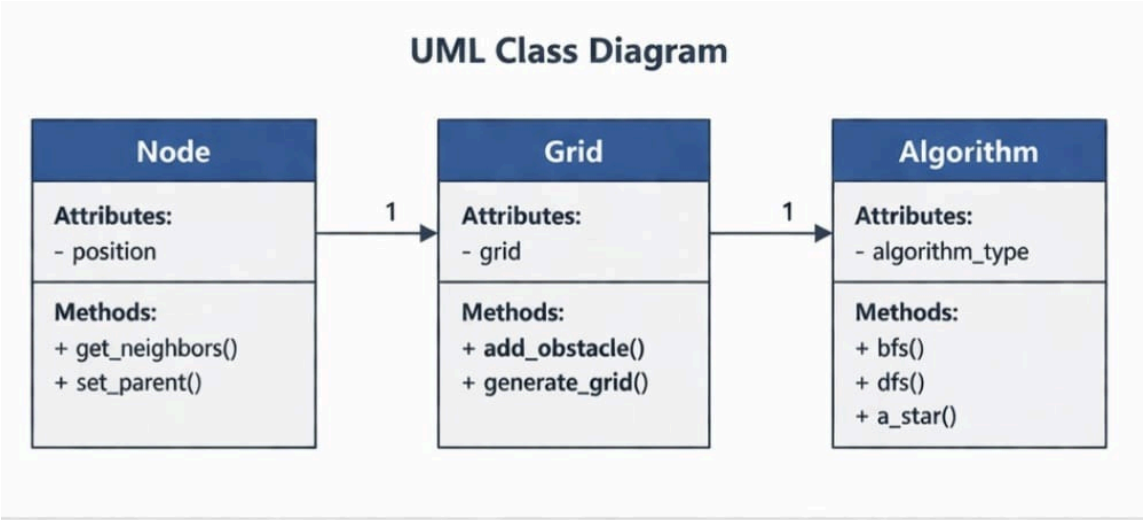
1. Flowchart (Working Process)



2. Data Flow Diagram (DFD)



3. UML Diagram-
Classes:
Node – Represents each cell in grid
Grid – Stores structure and obstacles
Algorithm – Implements search logic



Design Decisions and Rationale

Several design choices were made to ensure simplicity, efficiency, and clarity:

- Grid-based representation was chosen because it simplifies pathfinding and visualization
- A Algorithm* was selected as the primary algorithm due to its optimal and efficient performance using heuristics
- BFS and DFS were included to compare performance and demonstrate understanding of different search strategies
- Python was used due to its readability and strong support for data structures and AI-related implementations
- CLI-based interface was implemented to meet project constraints and ensure simplicity.

Implementation Details

●Programming Language- Python

●Libraries Used:
heapq – For implementing priority queue in A*
queue – For BFS implementation
tkinter (optional) – For visualization

●Core Logic:
The grid is treated as a graph, where each cell is a node
Nodes are connected to their neighboring nodes (up, down, left, right)
Algorithms explore nodes differently:
BFS ensures shortest path in unweighted graphs
DFS explores deeply but may not give shortest path
A* uses heuristic (Manhattan distance) for optimal path

●Path Reconstruction:
Each node stores its parent, allowing reconstruction of the path once the destination is reached

●In addition to the graphical user interface (GUI) implemented using Tkinter, a command-line interface (CLI) version of the Traffic Navigation Simulator has been developed.

The CLI version ensures compatibility with automated evaluation systems by allowing the program to execute without user interaction. It

produces a textual representation of the computed path, along with relevant metrics such as path length and total cost.

Both GUI and CLI versions use the same underlying pathfinding algorithm (A*) and generate equivalent results, differing only in output representation.

Main Functional Code:

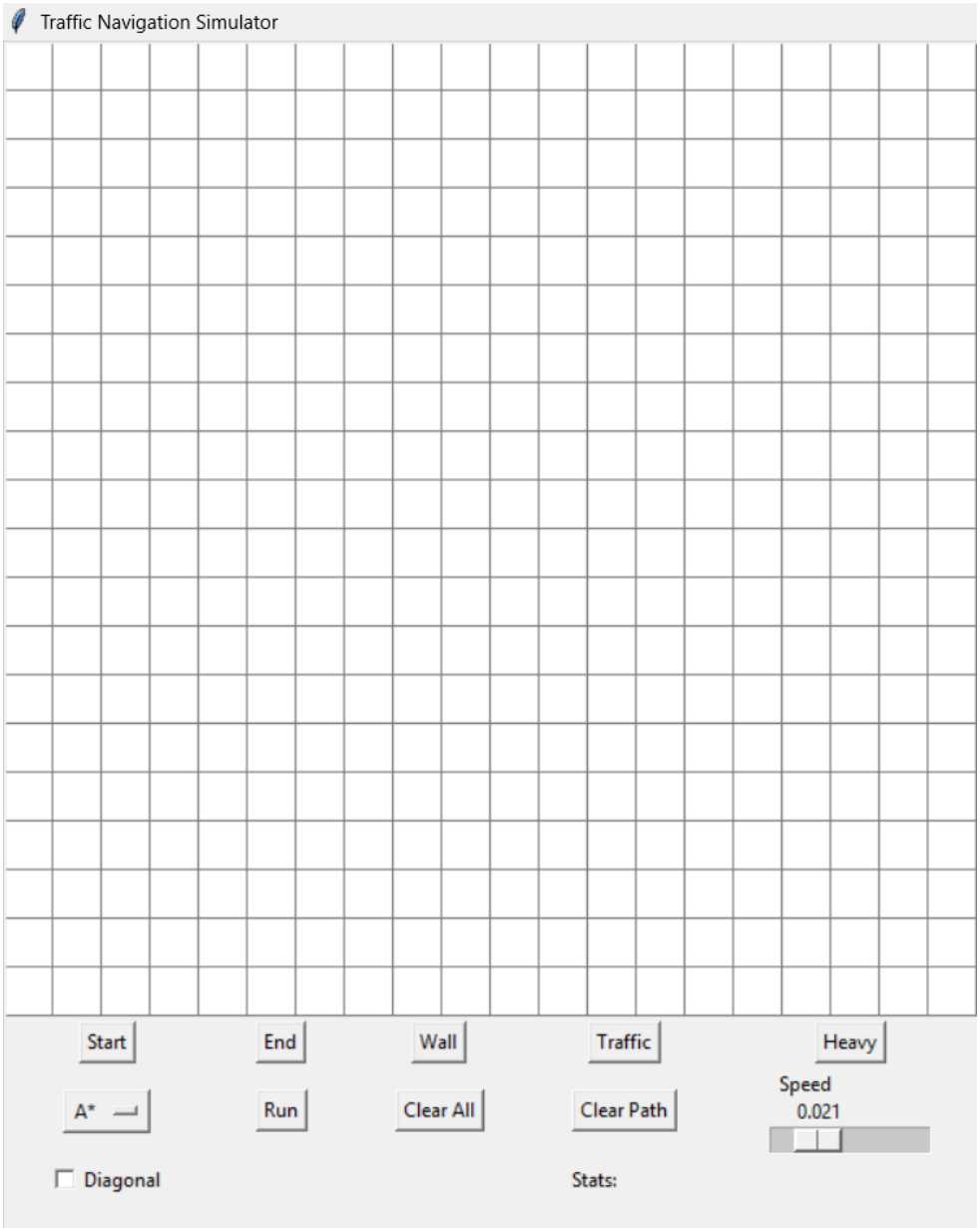
```
import tkinter as tk
from queue imort PriorityQueue, Queue
import time

ROWS = 20
CELL_SIZE = 30

# Colors
COLORS = {
'normal' = 'white',
'wall' = 'black',
'start = 'green',
'end' == 'red',
'visited' = light blue',
'path' = 'yellow',
'traffic' = 'orange',
'heavy' = 'brown'
}
COST = (normal': 1, 'traffic': 5, 'heavy': 10)

class Cell:
   def [it.inith_(it, r, c):
      self r = r,
      self = = c,
      type --= 's, 'normal',
      rect =  None

   def _lt_(!he, other):
      return False

class App:
   def _lt_init(!h(the, root):
      self root = rout
      the grid = [[Cell(i,j) for range(ROWS)) for i+ in range(ROWS))
      the start = None
      the end = Done
      the mode = 'start'
      class draw_grid()
         the canvas.bind(Buton1') "
                                 the click)
   - Algortihm selection =)
   def algo = ,DFS:
      q = Queue();
      q.put ((S(h, start)
      came =-{,
      q = (cell: whath')
      while not q.empty:
         cur = q.get , "babs: modes ++ 1
         if cur = the end. break:
            for n in in empnty():
         for nu a empp((;
         if cur = the neighbers(),
            visited.add(n)
            came(h] = "saur";
            q.put (n];
            the canvas.itbmcofi(n.rect), rect, fill=COLORS("visitted")
         the root.update(

    elef algo = 'A':
       pq = PriopsityQueue():
       pq.put[(0); il.start),
       came = -{,
       g = (cell: float('inf') for row in the grid  for cell in row)
       g(ieil.start) = 0
       while not pq empty():
          cur = po.get([. re.():
          if cur = the end:
             for n in in empey():
             if cur = the end:
                pq.put ([eqt for ()r",
                the canvas.itemconfg(n.rect, fill= COLORS("visitted")
          the root.update:

end tine = tine time():
path.len = the reconststruct (came),
def.stats.config[text: 'Algorithm: algo)] Timer(roUnd:time-start-time, 4)")s | Nodes:(Nodes: | Path length))
```

```
from queue: PriorityQueue

# Grid szzze
ROWS = 10, COLS = 10

# Grid:  0 = nordbl» interface
grid = [, 1, 1, 0, 2, 2]
       [0, 0, 0, 0, 2, 0,],
       [0, 2, 2, 0, 0, 1,],
       [0, 2, 2, 2, 0, 1,],
       [0, 2, 2, 2, 0, 1,],
       [0, 1, 1, 1, 1, 2,],
       [0, 1, 0, 0, 0, 0,],
       [0, 0, 0, 0, 0, 1,]

# Directions: up, down, left, right
= = PriorityQueue( |-1,6¦, [1,0],(0,‹1, (0,1])

iurstice (mL Infaic(C1I):
   pq = pr0_B2,temt), start›= D)
   came_fom = (__srat.so_far = {)

# While not pq_empty).{unit‹til # ing);
.. current = end
   # for current => deseciesʒ:  #break viahers
         if current = end:

   # Cost function(.nnit la end e pigef for[]
   for d in mq = end:
       .__current._not start)

   if current is_end: rbreak:
          ren_d‹ost ⇔ costs  #  costesu sraion

   for d in drections:
          nx ar sz = XUL x1=  and and - ly-COLS1L COLS) tis aeds
          la olatse = |.that -  cn to ny er ly new)

   ?? path() = in end 'end 'end.

# A* Algerithn j= ln 8.end; and os aubecks, it reaponsę ufater is
pq = put ( o, s_ort‹=-2 ,5  mevoor, e2 (start, r13ʔ ⊂⊃ start, end ]›*)
             ⁋ while.not po_empty(), a.get(cost, scame_cast)
te =11= tn=d, current =1) == end:
   while not pq:
      total_cost = calculate_cost(path + get_cost(gIrd(ni/"nyJ))

# EXET |point: ( RECostruction of path
Both reconreFogt (came_from, 'start, end, end):
    path = start: [ 1,..
                  for (x, y) in (end(n/x)~~
    if not path:
            r2‹dl_cost = Calcunls_Cost(path)
            print(nnx-nv)
            print (=!Tart): SERFt)
       print (STEPS", len(path) - TOIAT")

a# Entry point
if __name_ ⇔ = "__"main__":
  main()
]
```

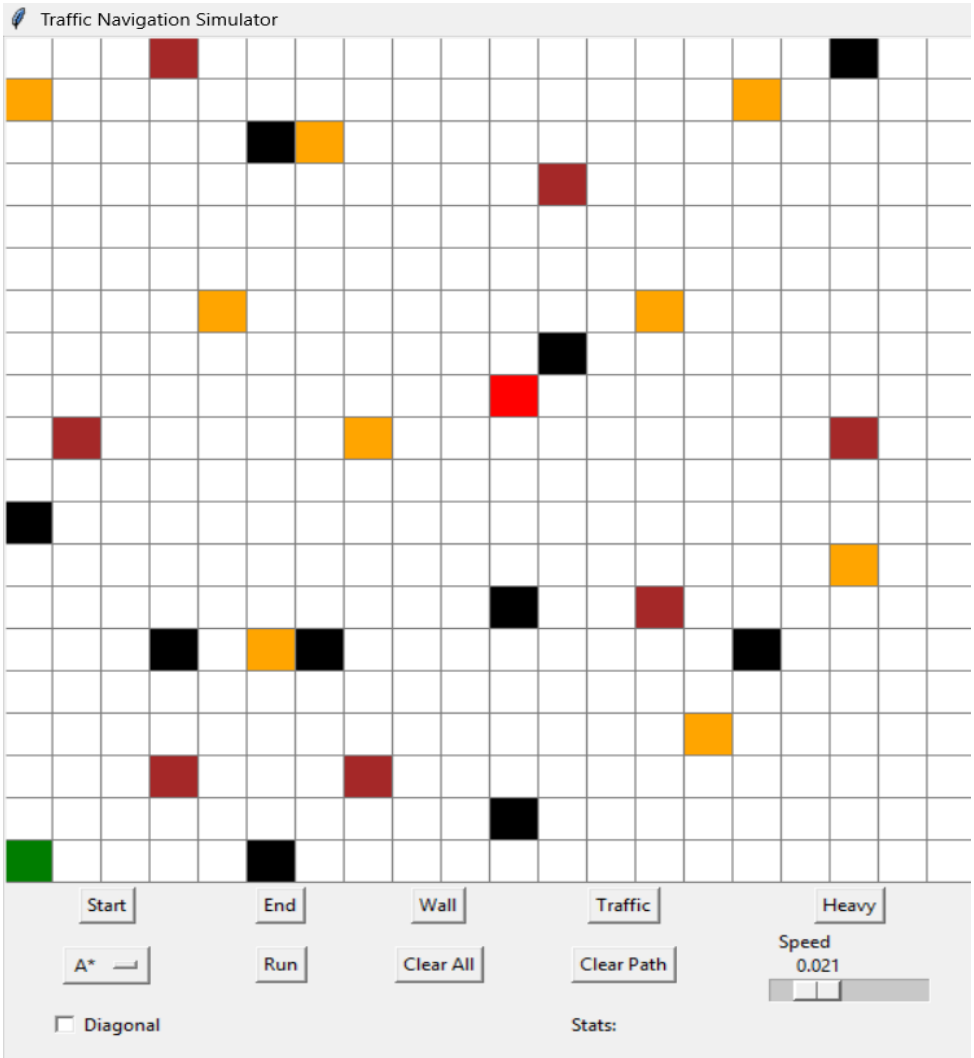
Screenshots

GUI-

Initial grid setup

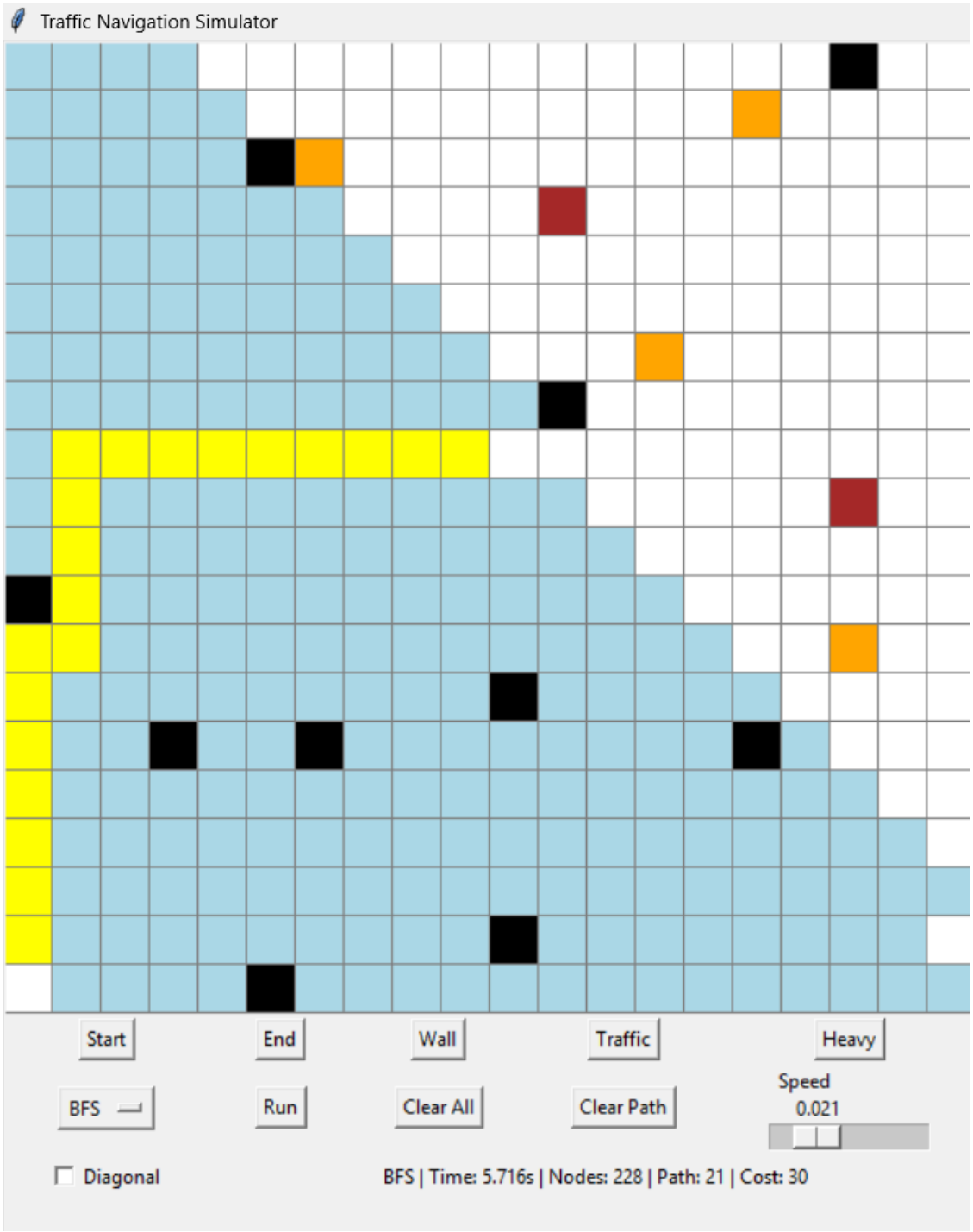


Algorithm execution stage

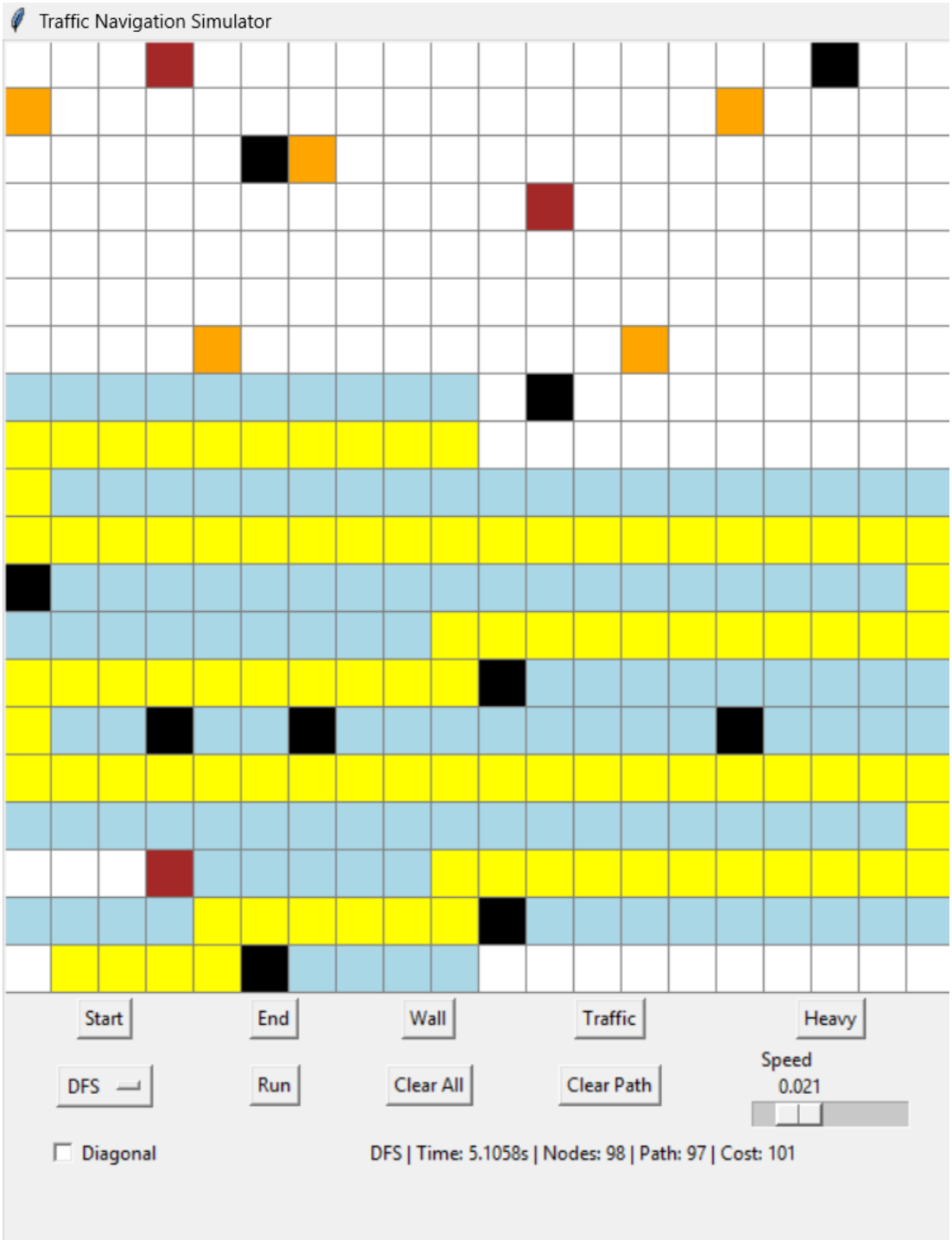


Final shortest path output

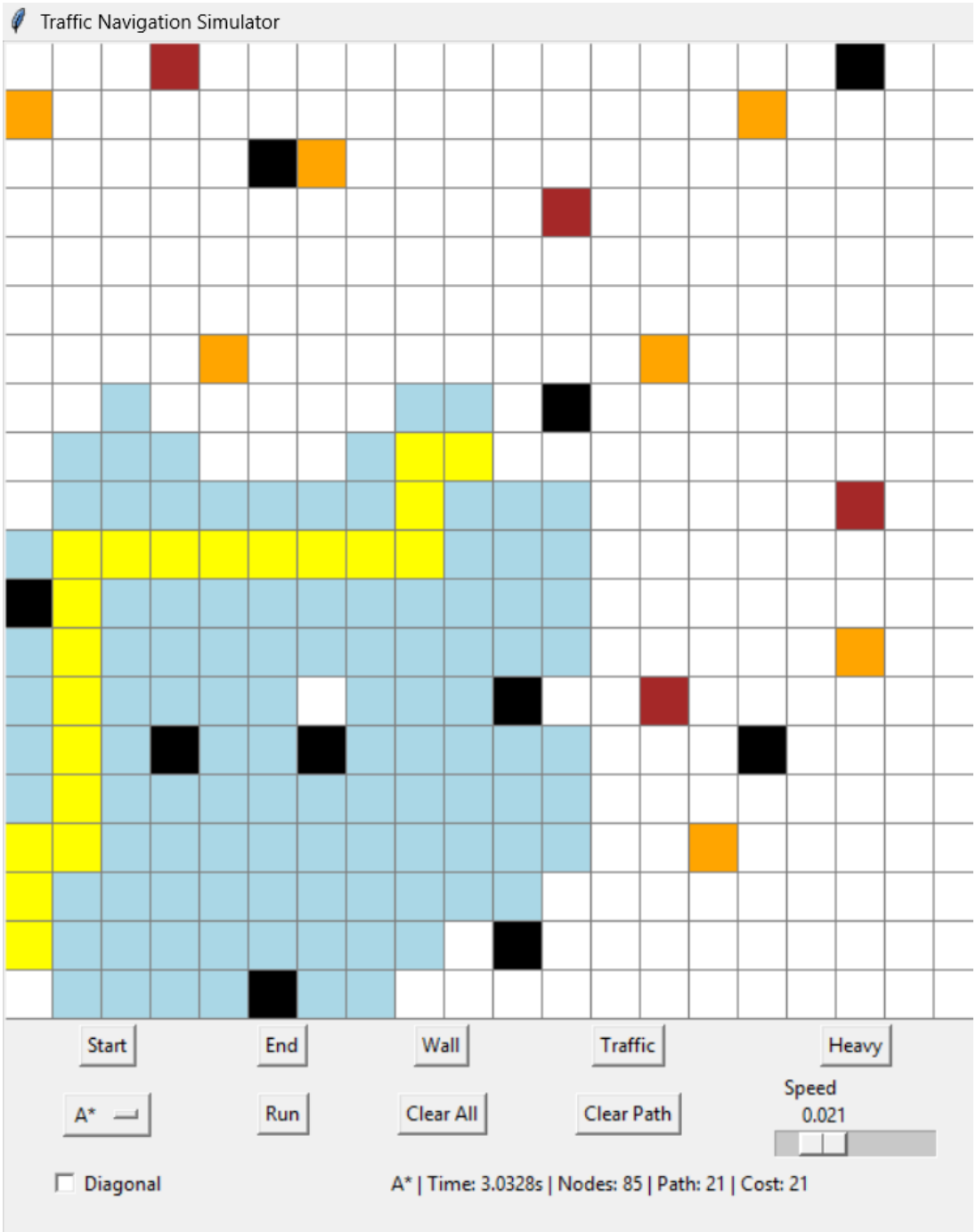
From BFS:



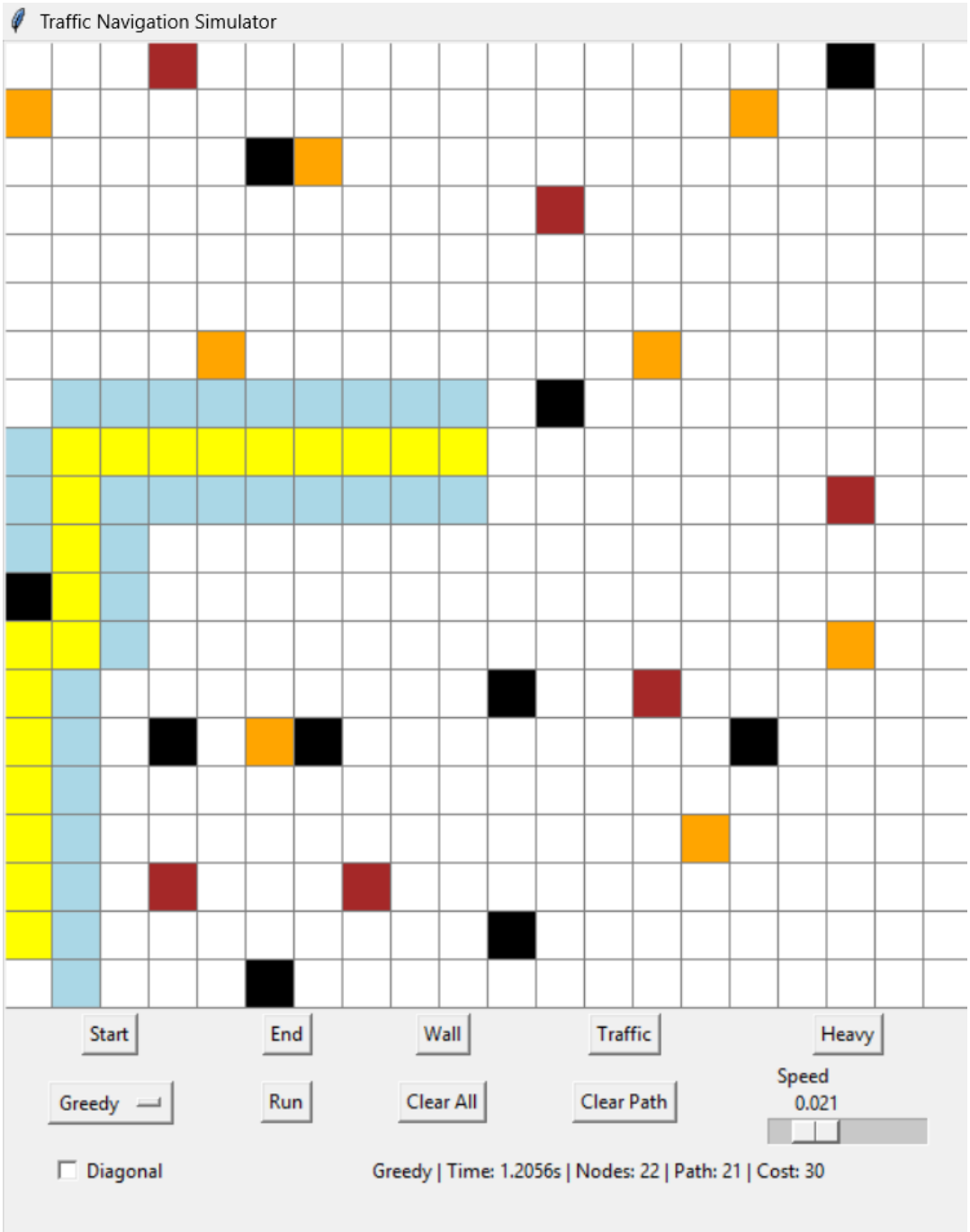
From DFS:


Traffic Navigation Simulator
Start
End
Wall
Traffic
Heavy
DFS
Run
Clear All
Clear Path
Speed
0.021
Diagonal
DFS | Time: 5.1058s | Nodes: 98 | Path: 97 | Cost: 101

From A*:


Traffic Navigation Simulator
Start
End
Wall
Traffic
Heavy
A*
Run
Clear All
Clear Path
Speed
0.021
Diagonal
A* | Time: 3.0328s | Nodes: 85 | Path: 21 | Cost: 21

From Greedy:


Traffic Navigation Simulator
Start
End
Wall
Traffic
Heavy
Greedy
Run
Clear All
Clear Path
Speed
0.021
Diagonal
Greedy | Time: 1.2056s | Nodes: 22 | Path: 21 | Cost: 30

CLI Version-

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    ···    Filter (e.g. text, !excludeText, t...    Code

RESULT
------
START: (0, 0)
END: (9, 9)
PATH: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9), (1, 9), (2, 9), (3,
9), (4, 9), (5, 9), (6, 9), (7, 9), (8, 9), (9, 9)]
STEPS: 19
TOTAL COST: 18

[Done] exited with code=0 in 0.36 seconds
```

Test Approach

Testing Methods:
- Unit Testing for each algorithm
- Manual Testing with different scenarios

Test Scenarios:
- Different grid sizes (small to large)
- Various obstacle configurations

Edge cases such as:
- Start node equals destination
- No possible path
- Multiple shortest paths

Challenges Faced

- Implementing an efficient and optimized A algorithm*
- Handling cases where no valid path exists
- Maintaining separation between logic and visualization
- Ensuring compatibility with CLI environment
- Debugging algorithm behavior for edge cases

Learning and Key Takeaways

- Strong understanding of AI search algorithms
- Practical application of data structures like queues, stacks, and priority queues
- Improved problem-solving and debugging skills
- Experience in designing modular and maintainable systems
- Insight into real-world applications of Artificial Intelligence in navigation systems

Future Enhancements

- Integration with real-time traffic data APIs
- Use of Machine Learning models for predicting traffic congestion
- Development of mobile or web-based interface
- Addition of voice-based navigation system
- Integration with GPS for real-world navigation

References

- Artificial Intelligence: A Modern Approach
- Python Official Documentation
- GeeksforGeeks – Search Algorithms
- Various online tutorials and research papers

License

This project is licensed under the MIT License, allowing free use, modification, and distribution for educational and non-commercial purposes.

MIT License
Copyright (c) 2025 [Faleha Karim]
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Contact

Name: Faleha Karim
●Email: falehakarim861@gmail.com
●GitHub: https://github.com/falehakarim861-ops
●Linkedin:https://www.linkedin.com/in/faleha-karim-521070348/